

**SENAI – SERVIÇO NACIONAL DE APRENDIZAGEM
INDUSTRIAL**

**RELATÓRIO DE BANCO DE DADOS
GERENCIAMENTO DE VENDAS**

Aluno: Wesley Thibes

Salto Veloso

2025

1. INTRODUÇÃO E CONTEXTUALIZAÇÃO

Uma empresa que trabalha com vendas precisa organizar corretamente suas informações para conseguir controlar seus clientes, produtos, pedidos e vendas. Quando esses dados ficam armazenados apenas em planilhas ou em diferentes arquivos, podem surgir problemas como informações duplicadas, dificuldade para encontrar registros, erros durante alterações e falta de controle sobre os dados.

Para solucionar esse problema, foi proposta a criação de um **banco de dados relacional utilizando SQL**. O objetivo é centralizar as informações da empresa em um único sistema, permitindo armazenar, consultar, alterar e excluir dados de maneira organizada e segura.

O banco de dados desenvolvido possui tabelas responsáveis por armazenar diferentes tipos de informações. Entre elas estão as tabelas de **clientes, produtos e pedidos**, que possuem relacionamentos entre si por meio de chaves primárias e estrangeiras.

Além das tabelas, foram utilizadas outras ferramentas importantes do banco de dados, como **Stored Procedures, Views e Triggers**. Esses recursos permitem automatizar tarefas, facilitar consultas e manter o controle das alterações realizadas nas informações.

Dessa forma, o projeto busca representar uma situação próxima da realidade de uma empresa, criando uma estrutura que poderia ser utilizada posteriormente por um sistema de vendas.

2. TIPO DE BANCO DE DADOS E JUSTIFICATIVA

O projeto utiliza um **banco de dados relacional**, implementado utilizando **SQL**, com o gerenciamento podendo ser realizado através do **MySQL/MariaDB**.

Um banco de dados relacional organiza as informações em **tabelas**, formadas por linhas e colunas. Cada tabela possui uma finalidade específica e pode se relacionar com outras tabelas.

Por exemplo, a tabela **clientes** armazena informações dos clientes, enquanto a tabela **pedidos** registra as compras realizadas. Dessa maneira, é possível relacionar um cliente aos seus pedidos.

Por que utilizar um banco de dados relacional?

A escolha desse tipo de banco de dados ocorre principalmente pela necessidade de manter os dados organizados e relacionados.

Entre suas principais vantagens estão:

- Organização dos dados em tabelas;
- Facilidade para realizar consultas;

- Utilização de chaves primárias e estrangeiras;
 - Redução da duplicação de informações;
 - Maior segurança e controle dos dados;
 - Possibilidade de utilizar relacionamentos entre tabelas;
 - Facilidade para gerar relatórios;
 - Possibilidade de utilizar Procedures, Views e Triggers.
-

3. TABELAS DO BANCO DE DADOS

As tabelas são uma das principais partes do banco de dados. Elas são responsáveis por armazenar as informações utilizadas pelo sistema.

Neste projeto, foram utilizadas tabelas relacionadas ao funcionamento de uma empresa de vendas.

3.1 Tabela Clientes

A tabela **clientes** tem como objetivo armazenar os dados dos clientes da empresa.

Nela podem ser armazenadas informações como:

- ID do cliente;
- Nome;
- CPF ou CNPJ;
- Rua;
- Número;
- Bairro;
- CEP;
- Referência;
- Telefone;

```
CREATE TABLE clientes (  
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    cpf_cnpj VARCHAR(14) NOT NULL,  
    rua VARCHAR(255),  
    numero VARCHAR(50),  
    bairro VARCHAR(100),  
    cep VARCHAR(10),  
    referencia VARCHAR(255),  
    telefone VARCHAR(20)  
);
```

Função da tabela:

Armazenar os dados cadastrais dos clientes e identificar cada cliente de forma única.

3.2 Tabela Produtos

A tabela **produtos** é responsável por armazenar as informações dos produtos comercializados pela empresa.

Entre os dados que podem ser armazenados estão:

- ID do produto;
- Nome;
- Preço;
- Estoque;
- Categoria;

```
CREATE TABLE produtos (  
    id_produto INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    preco DECIMAL(10,2) NOT NULL,  
    estoque INT NOT NULL,  
    categoria VARCHAR(50)  
);
```

Função da tabela:

Armazenar as informações dos produtos comercializados pela empresa.

3.3 Tabela Pedidos

A tabela **pedidos** armazena as informações referentes às compras realizadas.

Ela pode possuir informações como:

- ID do pedido;
- ID do cliente;
- ID do produto;
- Quantidade;
- Número do pedido;
- Data do pedido;

```
CREATE TABLE pedidos (  
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,  
    id_cliente INT NOT NULL,  
    id_produto INT NOT NULL,  
    quantidade INT NOT NULL,  
    num_ped VARCHAR(255),  
    data_ped DATETIME,  
  
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),  
    FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)  
);*/
```

Nessa tabela são utilizadas **chaves estrangeiras (Foreign Keys)** para criar os relacionamentos com outras tabelas.

Por exemplo:

pedidos → clientes

Um pedido pertence a um determinado cliente.

Também pode existir o relacionamento:

pedidos → produtos

Isso permite identificar qual produto foi comprado.

Função da tabela:

Armazenar os registros das compras realizadas, relacionando clientes e produtos.

4. RELACIONAMENTO ENTRE AS TABELAS

O relacionamento entre as tabelas é uma das principais características do banco de dados relacional.

Um exemplo de relacionamento utilizado no projeto é:

CLIENTES → PEDIDOS

Um cliente pode realizar vários pedidos, enquanto cada pedido pertence a um cliente.

Esse relacionamento pode ser representado como:

1 cliente → vários pedidos

Portanto, trata-se de um relacionamento **1:N (um para muitos)**.

Outro relacionamento importante é entre produtos e pedidos. Um produto pode aparecer em diversos pedidos, enquanto cada registro de pedido está relacionado a um determinado produto.

As **chaves estrangeiras** são utilizadas para estabelecer essas relações.

Essa organização evita a necessidade de repetir constantemente os mesmos dados e contribui para a normalização do banco.

5. STORED PROCEDURES

As **Stored Procedures**, também chamadas de procedimentos armazenados, são conjuntos de comandos SQL armazenados dentro do próprio banco de dados.

Elas podem ser criadas para realizar tarefas que são executadas com frequência.

Por exemplo, em vez de escrever várias vezes um comando **INSERT** para cadastrar um produto, pode-se criar uma Procedure responsável por realizar esse cadastro.

Uma Procedure pode receber informações através de **parâmetros** e utilizar esses valores para realizar uma determinada operação.

O que cada Procedure faz?

```
BEGIN
INSERT INTO produtos (
    nome,
    preco,
    estoque,
    categoria
) VALUES (
    p_nome,
    p_preco,
    p_estoque,
    p_categoria
);
END
```

A Procedure **sp_inserir_produto** é responsável por **cadastrar novos produtos no banco de dados**.

Ela recebe informações como **nome, preço, quantidade em estoque e categoria** e utiliza esses dados para inserir um novo registro na tabela **produtos**.

Em resumo:

Serve para facilitar e padronizar o cadastro de produtos no banco de dados.

```
BEGIN
  INSERT INTO pedidos (
    id_cliente,
    id_produto,
    quantidade,
    num_ped,
    data_ped
  ) VALUES (
    p_id_cliente,
    p_id_produto,
    p_quantidade,
    p_num_ped,
    p_data_ped,
    NOW()
  );
END
```

A Procedure **sp_inserir_pedido** é responsável por **registrar novos pedidos realizados pelos clientes**.

Ela recebe o **ID do cliente, ID do produto, quantidade e preço unitário**. Depois, insere essas informações na tabela **pedidos**.

A data do pedido é preenchida automaticamente utilizando **NOW()**, que registra a **data e hora atual**.

Em resumo:

Serve para registrar as compras realizadas pelos clientes, relacionando o cliente ao produto comprado.

```
BEGIN
INSERT INTO fornecedores (
    nome,
    cpf_cnpj,
    rua,
    numero,
    bairro,
    cep,
    telefone
) VALUES (
    p_nome,
    p_cpf_cnpj,
    p_rua,
    p_numero,
    p_bairro,
    p_cep,
    p_telefone
);
END
```

A Procedure **sp_inserir_fornecedor** é responsável por **cadastrar novos fornecedores** no banco de dados.

Ela recebe informações como **nome, CNPJ, telefone e endereço** e insere esses dados na tabela **fornecedores**.

Em resumo:

Serve para facilitar o cadastro e a organização dos fornecedores da empresa.

Vantagens das Procedures

As Procedures ajudam a:

- Reutilizar comandos;
- Evitar repetição de código;
- Padronizar operações;
- Facilitar a manutenção do banco;
- Automatizar tarefas.

Portanto, elas funcionam como uma espécie de **rotina armazenada dentro do banco de dados**.

6. VIEWS

As **Views**, ou visões, são consultas SQL armazenadas que podem ser utilizadas como se fossem tabelas virtuais.

Elas são muito úteis quando uma consulta possui vários comandos **SELECT**, **JOIN**, **GROUP BY**, **SUM**, **COUNT** ou outras operações.

Neste projeto, as Views são utilizadas principalmente para facilitar a análise das vendas.

View de clientes que mais compraram

```
CREATE VIEW view_clientes_que_mais_compraram AS
SELECT
    c.id_cliente,
    c.nome,
    COUNT(p.id_pedido) AS total_compras
FROM clientes c
INNER JOIN pedidos p
    ON c.id_cliente = p.id_cliente
GROUP BY
    c.id_cliente,
    c.nome
ORDER BY total_compras DESC;
```

Essa View pode apresentar informações como:

- Nome do cliente;
- Quantidade de compras realizadas;
- Total de pedidos.

Quais clientes realizaram mais compras?

id_cliente	nome	total_compras
1	João Silva	14
2	Maria Oliveira	14
3	Carlos Pereira	14
4	Ana Santos	14
5	Empresa ABC Ltda	14

View do produto mais vendido

```
CREATE VIEW view_produto_mais_vendido AS
SELECT
    pr.id_produto,
    pr.nome,
    SUM(p.quantidade) AS quantidade_vendida
FROM produtos pr
INNER JOIN pedidos p
    ON pr.id_produto = p.id_produto
GROUP BY
    pr.id_produto,
    pr.nome
ORDER BY quantidade_vendida DESC;
```

Outra View pode ser utilizada para identificar os produtos que tiveram maior quantidade de vendas em determinado período.

Ela pode apresentar:

- Produto;
- Quantidade vendida;
- Período da venda.

Essa consulta permite identificar:

Qual foi o produto mais vendido no último mês?

id_produto	nome	quantidade_vendida
18	Batedeira Planetária Oster 750W	32
8	Headset Gamer HyperX Cloud II	30
13	Micro-ondas Electrolux 20L	30
1	Smartphone Samsung Galaxy A54	28
20	Ventilador de Mesa Arno Silence Force	28

View de faturamento por cliente

```
CREATE VIEW view_faturamento_por_cliente AS
SELECT
    c.id_cliente,
    c.nome,
    SUM(p.quantidade * pr.preco) AS faturamento
FROM clientes c
INNER JOIN pedidos p
    ON c.id_cliente = p.id_cliente
INNER JOIN produtos pr
    ON p.id_produto = pr.id_produto
GROUP BY
    c.id_cliente,
    c.nome
ORDER BY faturamento DESC;
```

Essa View permite calcular o valor total comprado por cada cliente.

Ela pode apresentar:

- Nome do cliente;
- Quantidade de pedidos;
- Valor total das compras.

Qual o valor total de vendas por cliente?

id_cliente	nome	faturamento
7	Mariana Souza	64.317,2
12	Juliana Alves	55.781,4
4	Ana Santos	52.776,2
2	Maria Oliveira	52.204,78
8	Tech Soluções ME	48.372,0

Assim, a empresa consegue identificar quanto cada cliente movimentou em vendas.

7. TRIGGERS

Os **Triggers**, também conhecidos como gatilhos, são comandos executados automaticamente pelo banco de dados quando ocorre determinado evento.

Os eventos mais comuns são:

- **INSERT;**
- **UPDATE;**
- **DELETE.**

Isso significa que o banco pode executar uma determinada ação automaticamente quando um registro é inserido, alterado ou excluído.

Trigger de estoque

Um exemplo utilizado no projeto é um Trigger relacionado aos pedidos e ao estoque.

Quando um pedido é inserido, o Trigger pode diminuir automaticamente a quantidade do produto disponível no estoque.

Por exemplo:

Se determinado produto possui:

Estoque = 50

E é realizado um pedido de:

Quantidade = 3

O Trigger pode atualizar automaticamente o estoque para:

Estoque = 47

Dessa forma, não é necessário alterar manualmente o estoque depois de cada venda.

Trigger de histórico

Também pode ser utilizado um Trigger para registrar alterações realizadas nos produtos.

Por exemplo, quando o preço ou outra informação de um produto é alterada, o Trigger pode copiar os dados antigos para uma tabela de histórico.

Esse recurso é importante porque permite saber:

- Qual era o valor antigo;
- Qual produto foi alterado;
- Quando ocorreu a alteração;
- Qual usuário realizou a alteração.

Assim, os Triggers ajudam na **automação e no controle das informações**.

8. CONSULTAS E ANÁLISES

O banco de dados também foi desenvolvido para responder às principais perguntas propostas no desafio.

Quais clientes realizaram mais compras?

Essa consulta utiliza as tabelas de clientes e pedidos para contar quantos pedidos cada cliente realizou.

O resultado pode apresentar o cliente e a quantidade de compras, permitindo ordenar os resultados do maior para o menor.

Qual produto foi mais vendido no último mês?

Essa consulta verifica os pedidos realizados durante o último mês e soma a quantidade vendida de cada produto.

O produto que possuir a maior quantidade será identificado como o mais vendido.

Qual o valor total de vendas por cliente?

Nesse caso, os pedidos são agrupados por cliente e os valores das compras são somados.

Assim, é possível saber quanto cada cliente gastou.

Quais pedidos ultrapassam determinado valor?

Essa consulta utiliza uma condição para encontrar apenas os pedidos cujo valor seja maior que determinado limite.

Por exemplo:

Pedidos acima de R\$ 500,00.

Essa informação pode ser utilizada para análises comerciais e identificação de compras de maior valor.

9. NORMALIZAÇÃO DOS DADOS

A normalização é uma técnica utilizada para organizar o banco de dados e diminuir a repetição desnecessária de informações.

No projeto, os dados são separados de acordo com sua finalidade.

Por exemplo:

Cientes → informações dos clientes.

Produtos → informações dos produtos.

Pedidos → informações das compras.

Essa divisão evita colocar todas as informações em uma única tabela.

Sem normalização, poderia existir uma tabela contendo repetidamente:

Nome do cliente + CPF + endereço + produto + preço + pedido.

Se o mesmo cliente realizasse 20 compras, seus dados poderiam ser repetidos 20 vezes.

Com a utilização de tabelas relacionadas, os dados do cliente ficam armazenados uma única vez e os pedidos utilizam seu ID para realizar a ligação.

Isso torna o banco mais organizado e reduz a possibilidade de inconsistências.
